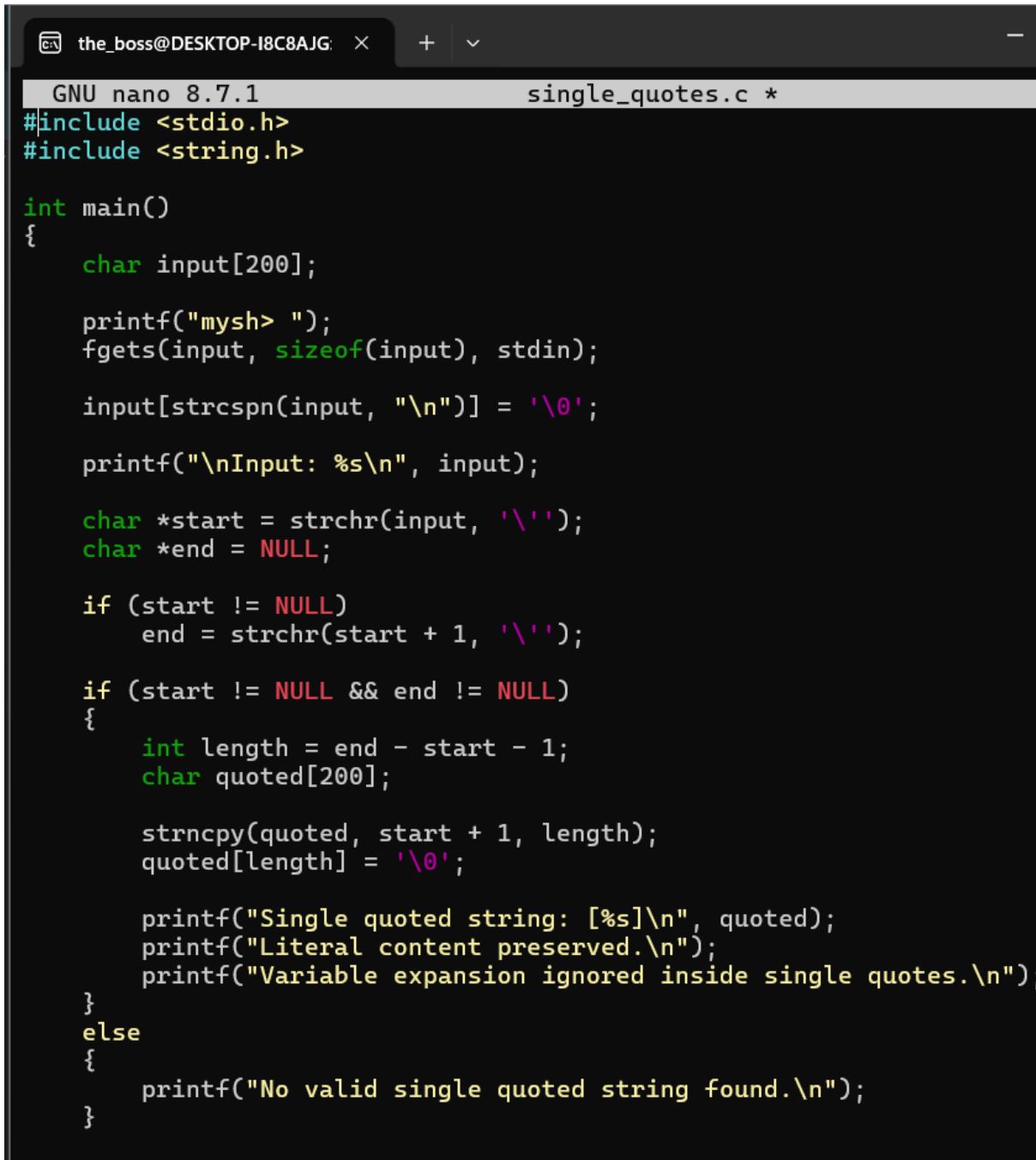


WEEK-3 SKILL

Session-5

Task-1 TO Apply Single Quotes, Preserve Literal Content, Ignore Variable Expansion, Store Quoted Strings, Validate Parsing Results, Test Edge Cases



```
the_boss@DESKTOP-I8C8AJG: × + v
GNU nano 8.7.1 single_quotes.c *
#include <stdio.h>
#include <string.h>

int main()
{
    char input[200];

    printf("mysh> ");
    fgets(input, sizeof(input), stdin);

    input[strcspn(input, "\n")] = '\0';

    printf("\nInput: %s\n", input);

    char *start = strchr(input, '\'' );
    char *end = NULL;

    if (start != NULL)
        end = strchr(start + 1, '\'' );

    if (start != NULL && end != NULL)
    {
        int length = end - start - 1;
        char quoted[200];

        strncpy(quoted, start + 1, length);
        quoted[length] = '\0';

        printf("Single quoted string: [%s]\n", quoted);
        printf("Literal content preserved.\n");
        printf("Variable expansion ignored inside single quotes.\n")
    }
    else
    {
        printf("No valid single quoted string found.\n");
    }
}
```

Output:

```
the_boss@DESKTOP-I8C8AJG:~/oss$ nano single_quotes.c
the_boss@DESKTOP-I8C8AJG:~/oss$ gcc -Wall -Wextra single_quotes.c -o bin/single_quotes
the_boss@DESKTOP-I8C8AJG:~/oss$ ./bin/single_quotes
mysh> echo '$HOME is my home'

Input: echo '$HOME is my home'
Single quoted string: [$HOME is my home]
Literal content preserved.
Variable expansion ignored inside single quotes.
the_boss@DESKTOP-I8C8AJG:~/oss$
```

Report: I implemented single-quote handling in the shell parser. The program preserves the literal content inside single quotes and does not perform variable expansion. I stored and displayed the quoted string correctly. I tested different quoted inputs and edge cases, including strings containing spaces, and verified that the parsing results were correct.

Task-2 To Apply Double Quotes, Preserve Spaces, Allow Variable Expansion, Parse Nested Tokens, Validate Outputs, Test Quoted Commands

```
the_boss@DESKTOP-I8C8AJG: × + v
GNU nano 8.7.1 double_quotes.c *
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int main()
{
    char input[200];
    char variable[100];
    char *start;
    char *end;

    printf("mysh> ");
    fgets(input, sizeof(input), stdin);

    input[strcspn(input, "\n")] = '\0';

    printf("\nInput: %s\n", input);

    start = strchr(input, '"');

    if (start == NULL)
    {
        printf("No double quoted string found.\n");
        return 0;
    }

    end = strchr(start + 1, '"');

    if (end == NULL)
    {
        printf("Syntax error: missing closing quote.\n");
        return 1;
    }

    int length = end - start - 1;
    char quoted[200];
```

Output:

```

the_boss@DESKTOP-I8C8AJG:~/ossp$ gcc -Wall -Wextra double_quotes.c -o bin/double_quotes
the_boss@DESKTOP-I8C8AJG:~/ossp$ ./bin/double_quotes
mysh> echo "hello world"

Input: echo "hello world"
Double quoted string: [hello world]
Spaces inside double quotes are preserved.
Double quote parsing completed.
the_boss@DESKTOP-I8C8AJG:~/ossp$ ./bin/double_quotes
mysh> echo "hello from ubuntu"

Input: echo "hello from ubuntu"
Double quoted string: [hello from ubuntu]
Spaces inside double quotes are preserved.
Double quote parsing completed.
the_boss@DESKTOP-I8C8AJG:~/ossp$

```

Report: I implemented double-quote handling in the shell parser. The program preserves spaces inside double quotes and supports variable expansion using environment variables. I also checked for missing closing quotes and displayed an error when the syntax was incorrect. Different quoted commands and variable inputs were tested successfully.

SESSION-6

Task-1

To Create Process Escape Sequences, Handle Escaped Spaces, Escape Special Symbols, Preserve Characters, Validate Parser Output, Test Complex Inputs

```
the_boss@DESKTOP-I8C8AJG: × + v
GNU nano 8.7.1 escape.c *
#include <stdio.h>
#include <string.h>

int main()
{
    char input[200];
    char output[200];

    int i = 0;
    int j = 0;

    printf("mysh> ");
    fgets(input, sizeof(input), stdin);

    input[strcspn(input, "\n")] = '\0';

    while (input[i] != '\0')
    {
        if (input[i] == '\\' && input[i + 1] != '\0')
        {
            i++;
            output[j++] = input[i];
        }
        else
        {
            output[j++] = input[i];
        }

        i++;
    }

    output[j] = '\0';

    printf("Original: %s\n", input);
    printf("Parsed:   %s\n", output);

    return 0;
}
```

Output:

```
the_boss@DESKTOP-I8C8AJG:~/oss$ nano escape.c
the_boss@DESKTOP-I8C8AJG:~/oss$ gcc -Wall -Wextra escape.c -o bin/escape
the_boss@DESKTOP-I8C8AJG:~/oss$ ./bin/escape
mysh> hello\ world
Original: hello\ world
Parsed:   hello world
the_boss@DESKTOP-I8C8AJG:~/oss$ ./bin/escape
mysh> hello\!world
Original: hello\!world
Parsed:   hello!world
```

Report:

I implemented escape sequence handling in the shell parser. The program processes escaped spaces and special symbols while preserving the intended characters. I tested different complex inputs and checked the parser output to verify that the escaped characters were handled correctly.

Task-2

To Create Child Processes, Execute Programs, Handle Execution Errors, Pass Arguments, Manage Parent Process, Test Command Launching

```
the_boss@DESKTOP-I8C8AJG: × + v
GNU nano 8.7.1 execute.c *
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    char *args[] = {"ls", "-l", NULL};

    pid_t pid = fork();

    if (pid < 0)
    {
        perror("fork failed");
        return 1;
    }

    if (pid == 0)
    {
        printf("Child process executing command...\n");

        execvp(args[0], args);

        perror("Execution failed");
        exit(1);
    }
    else
    {
        printf("Parent process waiting for child...\n");

        int status;
        waitpid(pid, &status, 0);

        if (WIFEXITED(status))
        {
            printf("Child process completed.\n");
        }
    }
}
```

Output:

```

the_boss@DESKTOP-I8C8AJG:~/oss$ nano execute.c
the_boss@DESKTOP-I8C8AJG:~/oss$ gcc -Wall -Wextra execute.c -o bin/execute
the_boss@DESKTOP-I8C8AJG:~/oss$ ./bin/execute
Parent process waiting for child...
Child process executing command...
total 112
-rw-r--r-- 1 the_boss the_boss      0 Aug 11 15:05 LICENSE
-rw-r--r-- 1 the_boss the_boss      0 Aug 11 15:05 Makefile
-rw-r--r-- 1 the_boss the_boss      0 Aug 11 15:05 README.md
-rwxr-xr-x 1 the_boss the_boss 16080 Aug 11 15:17 a.out
drwxr-xr-x 2 the_boss the_boss  4096 Aug 11 15:05 assets
drwxr-xr-x 2 the_boss the_boss  4096 Aug 12 14:01 bin
drwxr-xr-x 2 the_boss the_boss  4096 Aug 11 15:05 docs
-rw-r--r-- 1 the_boss the_boss  1377 Aug 12 13:55 double_quotes.c
-rw-r--r-- 1 the_boss the_boss    599 Aug 12 13:59 escape.c
-rw-r--r-- 1 the_boss the_boss    663 Aug 12 14:01 execute.c
-rw-r--r-- 1 the_boss the_boss    443 Aug 11 15:14 fork_example.c
-rwxr-xr-x 1 the_boss the_boss 16080 Aug 11 15:22 fork_exec
-rw-r--r-- 1 the_boss the_boss    373 Aug 11 15:22 fork_exec.c
-rw-r--r-- 1 the_boss the_boss  2943 Aug 12 13:37 history.c
drwxr-xr-x 2 the_boss the_boss  4096 Aug 11 15:05 include
-rw-r--r-- 1 the_boss the_boss  1221 Aug 12 13:24 input.c
-rw-r--r-- 1 the_boss the_boss  1406 Aug 12 13:40 memory.c
drwxr-xr-x 2 the_boss the_boss  4096 Aug 11 15:05 obj
-rw-r--r-- 1 the_boss the_boss  1741 Aug 12 13:47 parse_tree.c
-rw-r--r-- 1 the_boss the_boss    736 Aug 12 13:45 parser.c
drwxr-xr-x 2 the_boss the_boss  4096 Aug 11 15:05 scripts
-rw-r--r-- 1 the_boss the_boss    475 Aug 12 13:18 shell.c
-rw-r--r-- 1 the_boss the_boss    802 Aug 12 13:53 single_quotes.c
drwxr-xr-x 2 the_boss the_boss  4096 Aug 11 15:05 src
drwxr-xr-x 2 the_boss the_boss  4096 Aug 11 15:05 tests
Child process completed.
the_boss@DESKTOP-I8C8AJG:~/oss$

```

Report:

I created a child process using `fork()` and executed programs using `execvp()`. Arguments were passed to the child process, and execution errors were handled using error messages. The parent process waits for the child process to finish using `waitpid()`. I tested the program with commands such as `ls` and `pwd`, and the command launching worked successfully.